

算法学习手记

这是一份贯穿十章的学习手记从数组与链表出发经过栈队列哈希表树堆图最后以排序与复杂度回顾收束内容按章节顺序流动跨页不回头终章回顾在第十章等待每章以要开头以代码或对照表收尾全部例子都可以直接跑在本地。

第壹章 数组

数组是最朴素的容器连续内存常数下标访问缓存友好它的弱点同样出名中间插入与删除需要搬移元素代价与长度成正比选容器时先问访问模式再问修改模式答案多半自己浮现。

动态数组如C++的容量不足时按倍率扩容并把旧元素复制过去摊还分析说明尽管单次扩容是连续次追加的总代价仍是常数率是空间与时间的折中太小则频繁复制太大则浪费内存。

手记习惯写数组循环时先问一句遍历方向重要吗正向与反向不只是风格差异在删除元素的场景里直接决定正确性边遍历边删除是最常见的自伤动作倒序遍历或先标记后压缩才是稳妥做法。

数组还有一个常被忽视的性质它是几乎所有高级结构的地基字符串是字符数组，堆是藏在数组里的完全二叉树哈希表是桶数组加链表理解数组的行为就理解了一半的数据结构课。

索引的语义值得专门一提下标是偏移量而不是序号第一个元素在偏移零处把这一点内化之后半开区间的循环写法就不再需要背起点是偏移零终点是偏移恰好不含。

多维数组并不神秘它是数组的数组行的地址连续列的地址分散按行遍历与按列遍历的性能差距全部来自这条地址规律写嵌套循环时把大步长放在外层缓存会谢谢你。

稀疏数组是常被忽略的中间形态绝大多数位置为空时用坐标对加值的映射代替全量网格空间从平方降到线性棋盘表格时间轴都是稀疏化的受益者判断标准很简单非空占比低于一成就值得换表示。

数组与字符串的关系在C++格外密切字符串不可变按码单元索引行为

像一个冻结的数组理解了这一点动态语义就与数组完全对齐
两套只需记一套。

环形缓冲是数组的高级变体固定长度头尾指针取模推进写满即覆盖最旧音频
采样日志滚动限流窗口都靠它理解环形缓冲之后第节的队列可以用它实现
一个无锁版本。

数组题的第一直觉清单能否排序能否双指针能否前缀和能否哈希补一维四
连问下来八成的数组题都有了下手处清单本身不值钱值钱的是每次都真的问一
遍。

```
// dynamic array growth, amortized O(1) push
class Vec { constructor(){ this.a = []; }
  push(v) { this.a.push(v); } // engine doubles capacity
  get(i) { return this.a[i]; } // O(1) random access
  insert(i, v) { this.a.splice(i, 0, v); } } // O(n)
```

第节链表

链表用指针换连续性插入删除只改两个指针但随机访问必须从头走起单链表只
能向前双链表付出每节点一个额外指针的代价换来双向遍历环形链表则把尾部接
回头部用于轮转调度。

链表的经典错误是断链修改指针前没有保存后继节点从此失联稳妥的写法是先
画图再动笔先改新节点的指针再改旧节点虚拟头节点能消灭头结点特殊吗”
这一整类判断。

快慢指针是链表的配套技巧判断环找中点找倒数第几个都靠两个速度不同的
指针完成快指针每次两步慢指针每次一步环中相遇即有环走完未相遇即无环
。

链表与数组的对照可以浓缩成一张表访问插入删除内存三列各写一行把
这张表画在笔记第一页之后所有结构都照此格式记录复习时一目了然。

工程视角的补充语言运行时几乎从不裸用链表队列与栈由数组实现即可覆盖绝大
多数场景链表的价值在教学与面试它把指针操作逼到你眼前练的是手感不是
选型。

逆序是链表练习的必修课三指针滚动各司其职循环一轮挪一个
写完正向逆序再写区间逆序与一个一组逆序前者是后者的地基。

合并两个有序链表是另一门基本功双指针各牵一条谁小取谁尾插推进归并排序的合并阶段与它完全同构练熟这一题第8节的归并就只剩切分要写。

链表的哨兵技巧再怎么强调都不过分头让删除头节点与删除中间节点
答二为一尾让追加免去判空两个哨兵消灭一整类分支面试与评审都受益。

链表的内存代价常被低估每个节点除了数据还要背一个指针双链表背两个批量分配时节点散落各处缓存命中率远低于数组学习阶段画图即可工程阶段先测量再拍板。

有序链表的插入有讲究从头找位当然对维护一个跳表可以更快跳表用空间换高度把有序链表的查找拉到对数级别是有序集合的选择它属于第9节的树思想在链表上的投影。

```
// fast/slow pointers detect a cycle in O(n)
function hasCycle(head) {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next; fast = fast.next.next;
    if (slow === fast) return true; }
  return false; }
```

第9节栈与队列

栈是后进先出的单车道函数调用括号匹配撤销栈都是它的舞台队列是先进先出的排队机广度优先搜索离开它寸步难行两者都只暴露两端的操作约束越强，用途越明确。

用两个栈实现队列是经典练习入队压进栈出队时若栈为空就把栈全部倒入栈再弹出倒栈的时机是全部关键元素顺序恰好被翻转两次而复原摊还代价仍是常数。

优先队列则给排队机装上了插队规则堆顶永远是优先级最高者任务调度中的松弛候选合并的有序流背后都是同一个堆下一节展开堆本身。

括号匹配是栈的教科书案例左括号入栈右括号弹栈比对把三种括号混在一起，栈照样工作把引号与转义也混进来就需要状态机接力这也是栈够用到哪里天然分界线。

单调栈是栈的进阶用法栈内元素保持单调遇到破坏单调的元素就弹栈并结算下

一个更大元素柱状图最大矩形都是这个套路识别信号是题目里出现最近的
“一个更大的”。

栈与递归的关系值得点破递归就是隐式的栈系统替你压帧弹帧显式栈就是手写的递归深度优先搜索两种写法都行选择标准是深度是否可控中途是否需要保存现场。

队列的变体同样实用双端队列两端都能进出滑动窗口最大值靠它维持候选延迟队列给元素配一个可见时间到点才出队结构上的小改动换来语义上的大不同。

栈与队列的对称性在遍历里最漂亮深度优先配栈广度优先配队列，一行之差改变整个搜索的形状第节的图遍历会再次用到这对组合届时值得回来重读这段。

函数调用栈是栈概念的活教材调用即压栈返回即弹栈异常则一路弹到接住它的那一层调试器里的调用栈窗口就是本章概念的可视化现场值得打开看一次。

```
// two-stack queue, amortized O(1)
class Queue { push(v){a.push(v);} shift(){
    if (!b.length) while (a.length) b.push(a.pop());
    return b.pop(); } }
```

第节哈希表

哈希表用散列函数把键映射到桶数组平均的读写换来的是对散列质量的依赖。冲突处理有两条路拉链法把冲突者串成链表开放寻址让后来者另寻空位负载因子的警戒线通常画在四分之三。

负载因子超过阈值时扩容重散列新桶数组翻倍所有键重新落位重散列期间性能抖动明显实时系统里要提前预热或分批迁移渐进式重散列是工业级哈希表的标配。

键的设计要点是稳定相等的对象必须有相等的散列值否则昨天的缓存今天就取不出来不可变对象天然适合做键可变对象做键等于埋雷。

哈希表不维护顺序需要有序遍历时就退场让位给平衡树或跳表把快与有序都想要的场景先问一句顺序是否真的必需多数时候答案是否。

散列攻击是服务端的现实威胁构造一串散列到同一桶的键能把一次哈希查找拖成链表遍历带随机种子的散列与限长键是防御的两块砖学习结构时顺手了解攻防理解会深一层。

计数器是哈希表最常见的出场遍历一次边走边加词频统计两数之和的去重、图的邻接压缩第一行代码几乎都是

缓存是哈希表的第二战场以参数为键以结果为值递归加缓存等于记忆化缓存的三问命中键是什么失效条件是什么上限是多少在写第一行之前就要回答。

集合运算的加速也靠哈希判存在从扫描降为探测两个数组的交集因此变成线性把先建集合再查询当成条件反射,一半的暴力解法会自动消失。

哈希表与数组可以互相成全桶排序把值域切桶每桶一个小数组本质是用数组模拟哈希计数排序更是把哈希键退化成下标两个结构在本节和第节之间来回借力。

弱引用映射是哈希表的内存友好形态键不再被外部持有时表项自动可回收缓存类场景里手写淘汰策略省心代价是何时清不再受你控制。

```
// grouping with a Map: O(n) instead of O(n^2)
function groupBy(items, keyOf) {
  const groups = new Map();
  for (const it of items) {
    const k = keyOf(it);
    if (!groups.has(k)) groups.set(k, []);
    groups.get(k).push(it); }
  return groups; }
```

第节二叉树与遍历

二叉树的遍历有四个基本顺序前序中序后序与层序前三个用递归三行写完,层序借助队列逐层展开前序复制树中序读搜索树后序释放树各司其职。

递归在这里如鱼得水树的定义本身就是递归的中序遍历二叉搜索树得到升序序列这一性质是验证与调试的利器写完平衡操作之后跑一次中序序列不升序就说明旋转写错了。

平衡性决定性能最坏情况下的二叉搜索树退化成链表查找跌到红黑树与树用旋转维持平衡把最坏情况压回红黑以近似平衡换更少的旋转,工程上更常见。

线索化遍历迭代器写法都是在不用栈不用递归的约束下遍历树的手段约束本身未必必要但实现它们能加深对树结构的理解值得练一次。

前缀树和树用在字符串上公共前缀共享路径查找代价只与词长有关输入提示拼写检查路由表都是它的驻地前缀树是结构为数据形状服务的最佳例证。

递归深度等于树高平衡树的对数高度让递归安全退化树的高度让递归危险带着这条等式回头看第7节的快慢指针找中点两套工具就串成了一条线。

最近公共祖先是树上查询的代表题自底向上回溯遇到两侧都有命中的节点即为答案它与中序遍历一样考察的都是信息自下而上聚合的树直觉。

序列化与反序列化把树压平成字符串再还原前序加空标记是最简方案层序加计数是另一种两题互为逆运算,一起写完对树结构的掌握就闭环了。

树的路径问题是一族自顶向下求路径和自底向上求直径带转折的最长链套路都是左右子树各报一个数父节点聚合会一道就会一串。

二叉搜索树的操作全靠一个不变量左小右大对每个节点成立查找插入删除验证四件事都从不变量出发推删除带两个子节点时的找后继步骤是不变量最精巧的一次应用。

```
// three depth-first orders, one skeleton
function walk(n, out) {
  if (!n) return;
  out.push(n.v); walk(n.l, out); walk(n.r, out); // pre
  // walk(n.l); out.push(n.v); walk(n.r);    // in
  // walk(n.l); walk(n.r); out.push(n.v);    // post
}
```

第7节堆

堆是用数组存的完全二叉树父节点下标子节点与最大堆里父不小于子堆顶即全局最大数组下标与树形的互译是堆的第一课。

上浮与下沉是堆的两个基本动作新元素从尾部上浮到正确位置堆顶弹出后让尾部元素下沉重建堆序建堆可以自底向上完成优于逐个插入的。

堆排序就地完成建堆后反复弹顶放到数组尾部总复杂度 $O(n \log n)$ 缓存局部性不如快排实际常数偏大问题用小顶堆维护前k大是堆最实用的出场方式。

堆不维护全序兄弟之间没有可比性中序遍历堆没有意义需要第k大之外的

顺序问题时堆让位给排序或平衡树分清部分序够用与全序必需是选型的水分岭。

建堆的算法值得单独记一笔从最后一个非叶节点倒着下沉到底，一轮下来整棵树成堆它的节点数大于逐个插入的，看起来反直觉证明的关键是多数节点离叶很近。

优先级的动态更新是堆的软肋元素入队后优先级变了堆找不到它给元素存句柄配合位置索引表才能支持升键降键这也是为什么调度器宁可自己写双叉堆也不通用的原因。

中位数维护是双堆的招牌应用小顶堆存较大的一半大顶堆存较小的一半两堆顶夹出中位数数据流场景下每个元素只花对数时间面试与工程都常见。

堆的变体谱系顺便收录二项堆支持高效合并斐波那契堆把删边摊还压得更低配对堆以实践性能著称工程里二叉堆够用知道谱系是为了读论文时不迷路。

```
// heapify-down, the heart of a binary heap
function down(a, i) { for(;;) { const l=2*i+1, r=l+1; let m=i;
  if (l < a.length && a[l] > a[m]) m = l;
  if (r < a.length && a[r] > a[m]) m = r;
  if (m === i) return; [a[i], a[m]] = [a[m], a[i]]; i = m; } }
```

第节图的表示

图的两种主流表示邻接矩阵用的布尔阵查边快但空间邻接表每点挂一条邻居链表空间遍历邻居高效稀疏图选表稠密图选阵。

遍历两件套深度优先沿一条路走到底再回头天然适配递归与显式栈广度优先逐圈扩散配队列使用最先触达的距离即最短路无权图。

拓扑排序把有向无环图压成线性序反复摘掉入度为零的点若中途无点可摘而图中仍有剩余说明环存在依赖关系无法满足构建系统的依赖解析就是每天在跑的拓扑排序。

带权图交给动态规划者贪心加堆后者允许负边并负责检测负环两者的共同前提是理解松弛操作尝试经由让的距离估计更短。

连通分量与并查集是图论里的另一半世界不问最短只问是否连着并查集按秩合并加路径压缩单步摊还近似常数联通判定快得不像图算法。

网格图是图的第一应用现场每个格子是一个点四邻是边深度优先染一遍就是洪泛填充把二维下标压成一维编号邻接表就退化为四个偏移量代码短得不像图算法。

二分图判定用染色法从一个点出发交替染两色冲突即非二分匹配问题的入口大多在这里任务与工人课程与时段先判定二分再谈分配。

图的存储还有第三条路边集数组只存边的两端与权遍历效率最低但对这类按边排序的算法反而是最自然的输入结构没有优劣只有与算法的契合度。

最短路的实际工程版是星给一个到目标的估计函数估计不高估就能保持正确估计准就能少走弯路游戏寻路与地图导航都是它的日常现场。

```
// BFS on an adjacency list; distances in one pass
function bfs(adj, s) {
  const dist = new Map([[s, 0]]);
  for (const q = [s]; q.length;) {
    const u = q.shift();
    for (const v of adj.get(u) ?? [])
      if (!dist.has(v)) { dist.set(v, dist.get(u)+1); q.push(v); }
  }
  return dist; }
```

第8章 排序

排序算法的三问平均复杂度最坏复杂度是否稳定快排平均最坏时间复杂度是 $O(n^2)$ 并不稳定但需要辅助数组堆排序最坏有保障但不稳定三问答完选型自动完成。

工程实现从不只用一种小段落切换插入排序常数小次段落递归切分深度超限切换堆排序这种混合策略就是多数标准库的实际形态。

稳定性影响的是并列元素的相对次序按两个键排序时第二次排序若不稳定第一次的结果就会被部分打乱数据库的多列排序因此全部依赖稳定算法。

比较排序的下界是 $\Omega(n \log n)$ 源自决策树高度的论证计数排序基数排序绕开比较在线性时间内完成代价是对键值域的假设知道下界在哪里才知道什么时候该换赛道。

快排的基准选择决定命运固定取首元素在有序输入上退化为平方三数取中让最坏概率骤降随机取则把期望锁死在对数级第8章的散列攻击与这里是同一个道理：对手会瞄准你的确定性。

```
// ten sorts; master three first: quick, merge, heap
quickSort(data, 0, data.length - 1);
mergeSort(data, temp, 0, data.length - 1);
heapSort(data);
```

第9节复杂度回顾

复杂度分析的目的是预测规模增长时的行为曲线而不是精确计时常数因子交给实测增长趋势交给大两套工具各管一段混用就会得出啥希永远最快之类的错误结论。

摊还分析值得单独一节动态数组追加并查集按秩合并单步可能昂贵序列整体便宜看总账不看单步把这一条与第9节第9节对上三处摊还案例互为印证

。

对数的感觉要专门培养:一百万个元素的平衡树查找只有二十层,一千万也只有二十三层规模涨十倍对数只涨三步这就是分治结构敢于面对大数据的底气。

空间复杂度的三个常见来源递归栈辅助数组输出本身前两个可以优化改造代原地交换第三个是题目自带的别错怪自己把三栏并排写在第9节笔记边上复查时一目了然。

第九节收束下页的术语对照表是全书的浓缩版复习时先默写右列再回读左列。

```
// glossary: term, English, one-line note
```

数组

链表

栈

队列

哈希表

平衡树

堆

邻接表

拓扑排序

摊还

第9节终章回顾

终章回顾整条学习线数组与链表给了线性结构栈与队列给了访问次序哈希表给了平均常数树与堆给了对数台阶图把一切连成网络排序与拓扑排序提供了加工手段。

终章回顾的意义在于收束十个章节不是十个孤岛而是同一门手艺的十次打磨下次再遇到新结构先问它落在哪一层台阶上多数问题就有了答案。

回顾的另一个收获是看见呼应数组的摊还并查集的摊还快排的随机化哈希的随机化同一思想在不同章节各出场一次手记读到第二遍主线才会浮出来。

给后来者的练习清单放在最后数组原地去重链表逆序两栈队列缓存二叉树层序遍历课程表拓扑排序颜色分类三指针八道题对应八个章节做不出来就回读对应章节。

练习的验收标准不看题解写出用随机数据对拍给边界各造一个失败样本三步全过才算掌握只过第一步的属于眼会手不会”

写手记这件事本身也值得一句回顾每章要点加代码加反例的三段式让复习时间压到初读的三分之一结构化不是为了好看是为了将来能快速找回。

下一轮学习的入口已经想好把本手记的八个练习各写一遍测试让每一题都留下可回归的资产从读到写的跨越才是手记真正的终点。

手记完愿每次终章回顾都比上一次更快。